

(user) authentication

User authentication

→ Prove you are really the one you claim to be

⇒ Do not confuse:

→ **Identification** → **show** your «digital identity» (email, ID)

→ **Authentication** → **prove** your digital identity is controlled by you (e.g. you know a secret password)

→ Actually, not so simple...

⇒ There might be no requirement to link identifier to a real-life identity

→ Successful authentication = I'm reasonably assured that **the subject accessing the service today is the same as the one which accessed the service in the past.**

⇒ NIST SP 800-63-3 definition for digital user authentication: **the process of establishing confidence in user identities that are presented electronically to an information system**

Authentication Means

→ **Something you (and only you) know**

⇒ Password, PIN, secret key, answer to a pre-arranged question, ...

→ **Something you (and only you) have**

⇒ Smart card, Physical devices (tokens)

→ Very interesting area: “authentication” based on HW uniqueness → **PUF (Physically Unclonable Function)**

→ **Something you are (static biometrics)**

⇒ retina, fingerprint, face, ...

→ Privacy? And what about DNA (Remember Gattaca?)?

→ **Something you do (dynamic biometrics a.k.a. behavioral authentication)**

⇒ Voice recognition, hand writing, typing characteristics

→ ... even Mouse Movements!!

But there are also other specific means harder to classify, e.g. location-based auth

Authentication: “proof of...”

→ To “prove” I know a password does NOT necessarily imply revealing it

⇒ Though revealing the password is ONE POSSIBLE approach

→ Passwd Authentication Protocol - PAP

→ Different techniques → different levels of information disclosure

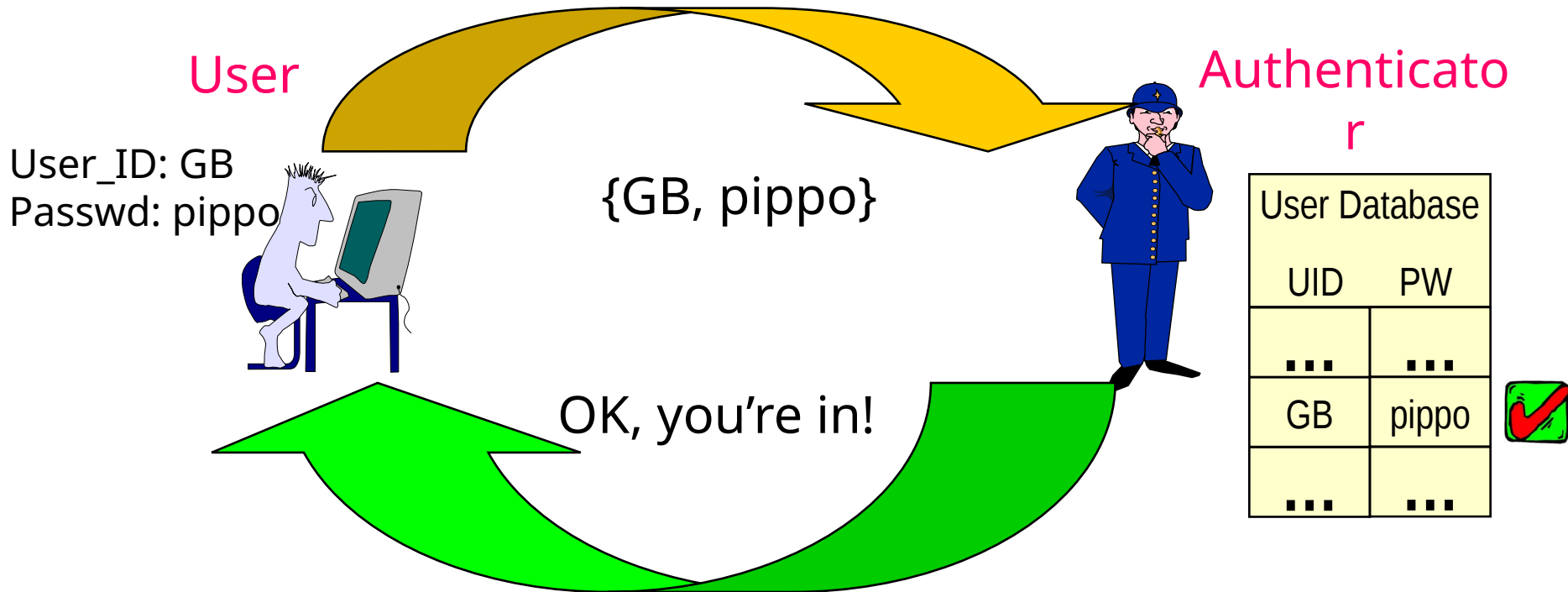
⇒ PAP = full disclosure

⇒ CHAP = some information leaks

⇒ ZKP = no information leakage

User authentication – recap: Password Authentication protocol

Prove you know your password by... showing it in clear!

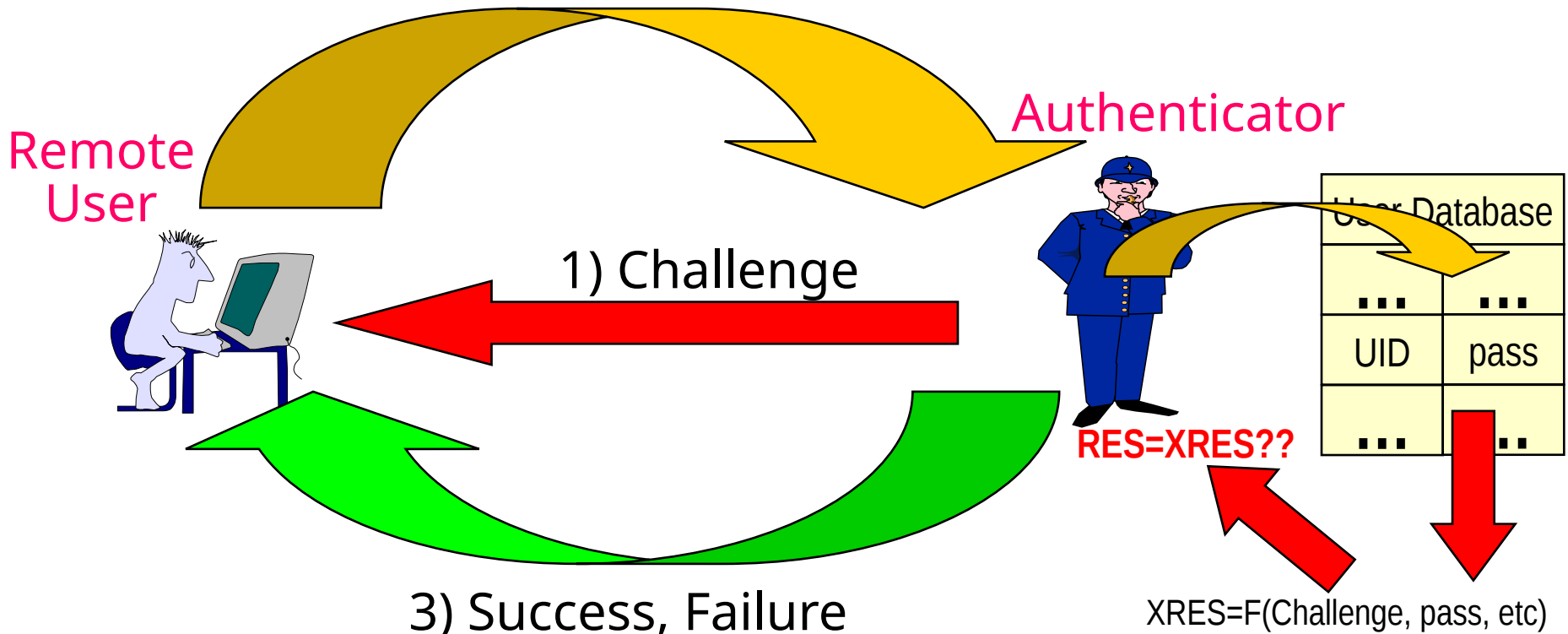


User authentication – recap:

Challenge-Handshake Auth Protocol (CHAP)

Prove you know your password by... sending the result of a secure computation!
(usually hash function but not necessarily – more later)

2) $UID, RES = \text{SECURE_FUNCTION}(\text{Challenge}, \text{Pw}, \text{etc})$



→ **Secret passwd never transmitted in clear - Hash is the usual approach, but it is NOT the only one**

→ Other common approach: encrypt challenge using P as secret key. But requires more care! More later.

Remark: Password or secret key?

→ Authentication

⇒ Prove I know a password or a secret

→ Password or secret? conceptually the same

⇒ A password is (should be) a secret

→ Practically there is a HUGE difference

⇒ Secret key: a random string of N bit
→ Probability to guess it = 1 out of 2^N

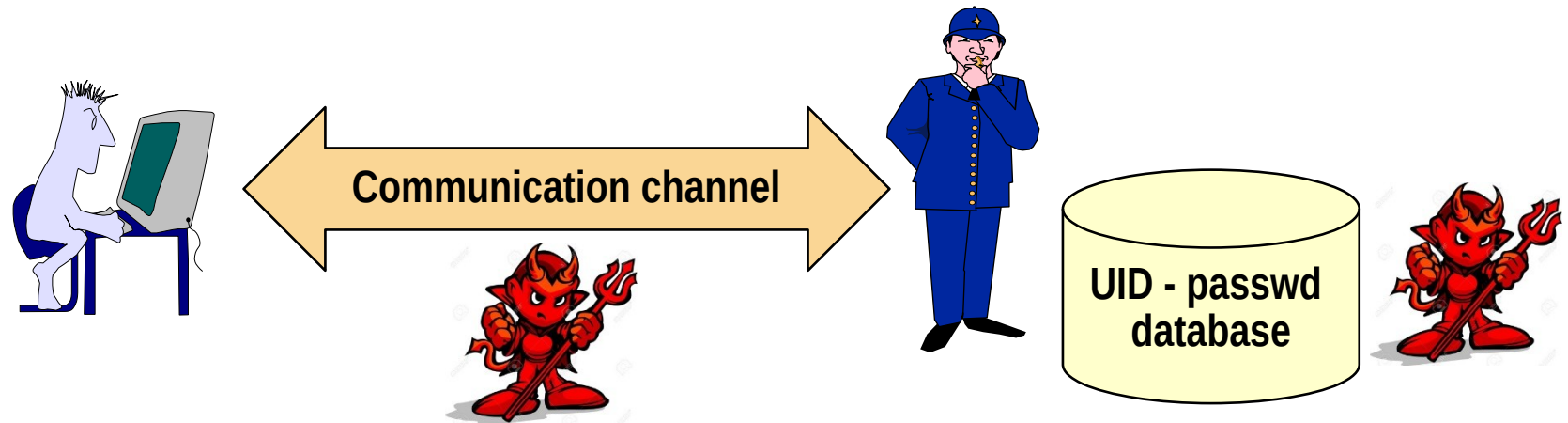
⇒ Password: a **low-entropy** string!!
→ Probability to guess it:
MUUUUCH lower than $n = 1$ out of 2^N

What is best? CHAP or PAP?

Correct answer: DEPENDS!!!

(depends on what? On the attacker's model)

2 main (*) attack models



→ Attack to the communication channel

⇒ Eavesdropper, MITM, replay, ...

⇒ Game Over!

→ If you want to use PAP, you MUST protect communication channel

→ E.g. online login via https, EAP-TTLS, etc

→ Attack to the backend UID-passwd database

⇒ Attacker penetrates system and steals entire passwd DB

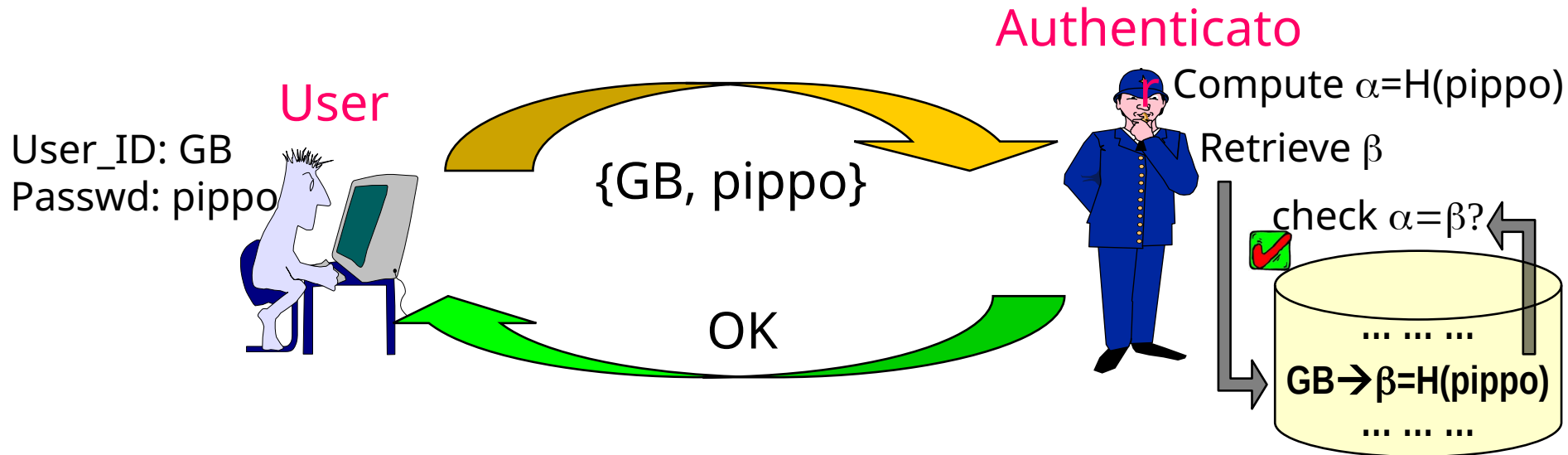
(*) other system level attacks: insiders, workstation hijacking, key loggers, fake screens, etc

⇒ Mitigation: hashed password database!

===== Giuseppe Bianchi =====

Hashed passwd database in PAP

➔ **Idea: store $H(\text{passwd})$ instead of passwd**



➔ **Improved security: if attacker steals DB, still has to «invert» passwd**

⇒ Good hash = one-way → only method is brute-force/guess
→ **Security ultimately depends on choosing «strong» passwd**

(actually: «salted» hash)

User	Salt	Hashed passwd = H(salt, passwd)
Giuseppe	a31	d8578edf8458ce06fbc5bb76a58c5ca4
Letizia	117	a00e014ec1a1148986e05d0c151b92c4
Ciro	1e8	f1ce609eb6591ecd3dc405282fef22f9
Jasmine	f7d	5b4805eae94d66c1a422189a53f9f06a
... ..		

→ Security: same as hashed passwd

⇒ Salt in clear: not nearly an issue for crypto hash!

→ advantages

⇒ Harder to crack against rainbow tables! (need one table per each salt value)

→ 12 bit salt = 2^{12} tables, huge memory requirements!

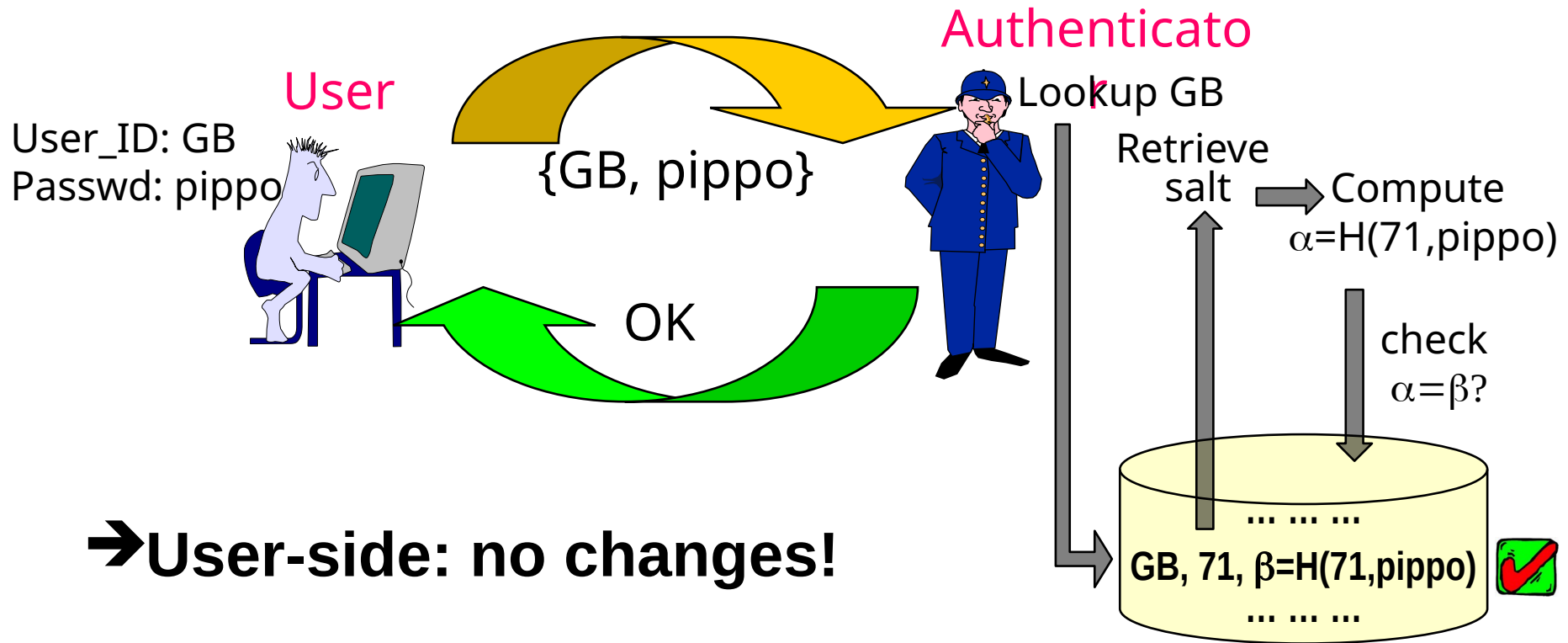
⇒ Same passwd but different salt → different hash

→ Attacker cannot see if passwd is reused – two cases

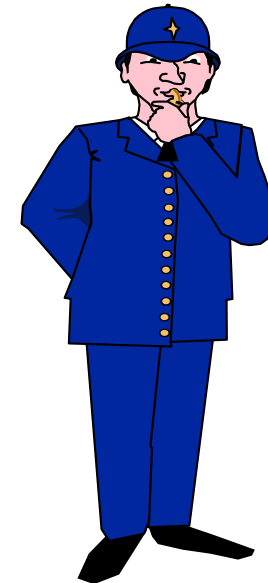
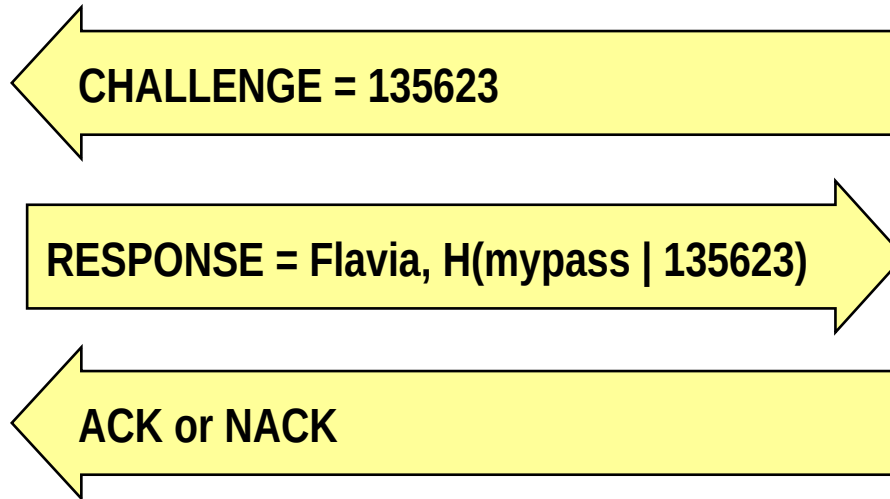
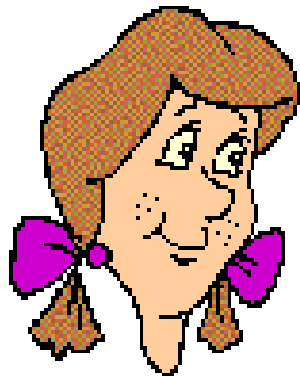
» Inside DB: same passwd among different users

» Across DBs: passwd reused by same user

Authentication using salt



And what about CHAP?



User Database	
...	...
flavia	mypass
...	...

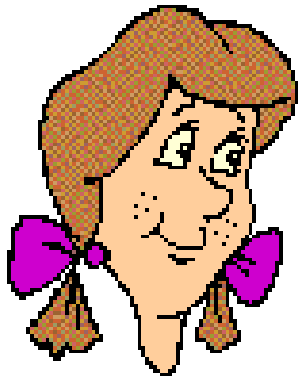
- No way to compute $H(\text{pw}, \text{challenge})$ without having pw in clear!
- Consequence: user DB must remain in clear → straightforward target for attack!
- **Conclusion: CHAP is worse than PAP against a backend attack model!!**

Take home:

it all depends on attacker's model!

- ➔ **In security there is NO «best» algorithm**
 - ⇒ **Solutions must be adapted to threat model**
- ➔ **If Attacker = eavesdropper, then CHAP better**
- ➔ **But if Attacker = backend, then PAP better!**
- ➔ **Hence there is not a «single» solution, but all depends on which attack you aim at defending**

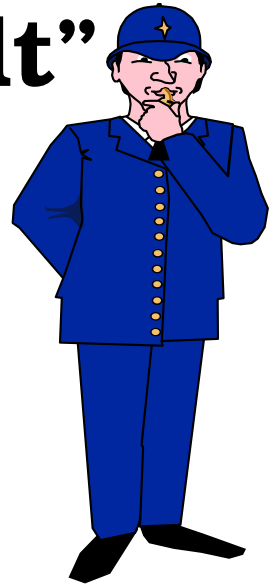
CHAP Mitigation: (explicit) “salt”



CHALLENGE = 135623; SALT = 9876

RESPONSE = Flavia, $H(H(9876, \text{mypass}) \mid 135623)$

ACK or NACK



→ Same idea as «usual» salt

- ⇒ Salt value is in clear
- ⇒ Store $H(\text{salt}, \text{passwd})$

→ very different reason and use!

- ⇒ Attacker cannot anymore «reuse» stolen passwd!
 - **But of course passwd strength and dictionary attacks still hold...**
- ⇒ After a breach (or periodically), regenerate entire DB using new salt

User Database: SALT=9876

...	...
flavia	$H(9876, \text{mypass})$
...	...

Hashed passwd: which hash?

➔ **Best crypto hash today: SHA256**

⇒ Seems very good idea to use it...

⇒ **Sure???**

⇒ **Why warning?**



Hashed passwd again: which hash?

SHA256: 2 problems!

→ 1. Designed to be fast

→ Fast computation = faster cracking!

→ 2. Used in Bitcoin mining

→ Today: specialized and cheap
ultra-high performance HW
for SHA256 computation!

→ SHA256 hash rates

⇒ PC: core i7 3930k

→ 66.6 MH/s (~190watts)

⇒ Hardware (e.g. Antminer): @ July 2023:

→ Bitmain AntMiner S19j pro: 118 TH/s (~3 kW, 2259 EUR)

→ Bitmain AntMiner S19j XP: 151 TH/s (~3 kW, 3999 EUR)

» **0.93 seconds average time to brute force 8 bytes truly random base64 charset passwd!**

» **About 22 seconds for 8B passwd using all the keyboard chars/symbols**



Hashed passwd: best with slow hash!

→ **Bcrypt: designed in 1999 for unix (OpenBSD) systems**

⇒ SPECIFICALLY (!) for password hashing

→ **Includes salt + «cost» parameter**

⇒ Cost = x → requires 2^x iterations - example:

» cost = 10 → o(1000) iterations per hash

» Cost = 13 → o(10.000) iterations

→ **Performance**

→ From: 2014, <https://www.usenix.org/system/files/conference/woot14/woot14-malvoni.pdf>

Cost / Device	12	10	8	5
Epiphany 16	9.64 c/s	38.7 c/s	151.3 c/s	1207 c/s
Zynq-7020	64.83 c/s	253.1 c/s	932.6 c/s	4571 c/s
Zynq-7045	226.3 c/s	888.6 c/s	3371 c/s	20538 c/s
HD 7970	35.76 c/s	142.9 c/s	569.2 c/s	4556 c/s
FX-8120	42.93 c/s	171.2 c/s	680.2 c/s	5275 c/s
Xeon Phi 5110P	50.18 c/s	200.7 c/s	800.8 c/s	6285 c/s
i7-4770K	53.67 c/s	214.2 c/s	852.8 c/s	6615 c/s

Take home

Again!

Your choice depends on the attacker model!!

→ **Attacker model against hashed passwd DB?**

⇒ Brute force, dictionary

→ **But hash functions designed for something else!**

⇒ collision resistance, pseudorandomness

→ **So? The stronger and best hash for us is a SLOW one!**

⇒ Use a programmatically SLOW hash functions, e.g. classic Unix' BCrypt

→ Include a cost parameter!!

» cost = 10 → o(1000) iterations per hash

» Cost = 13 → o(10.000) iterations

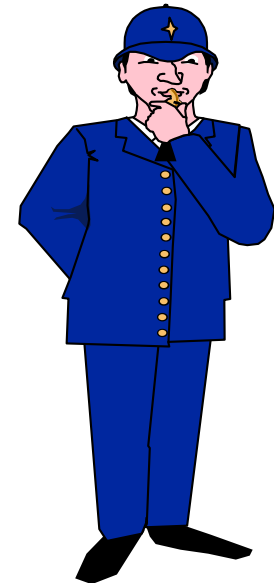
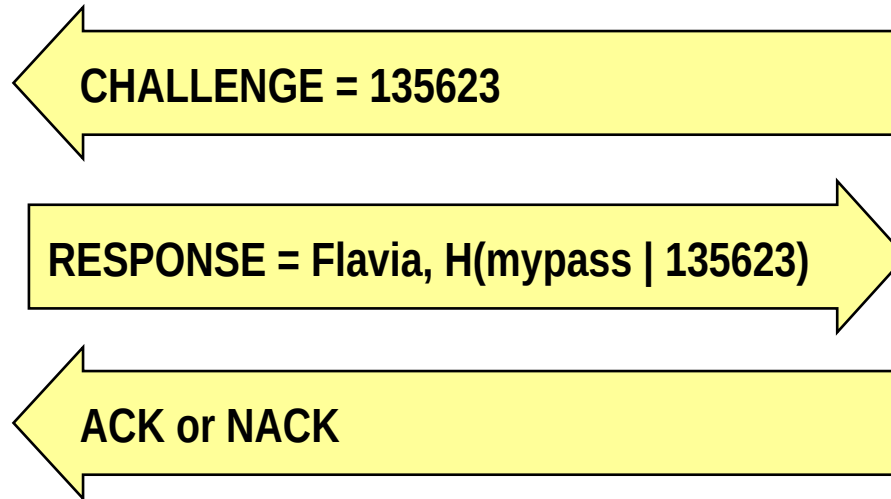
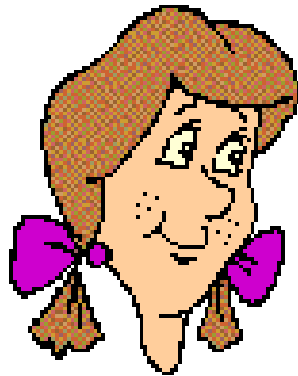
⇒ Newer passwd-specific hash constructions: Argon2, PBKDF2, etc

Mutual Authentication with CHAP

Take home of this lecture (again):

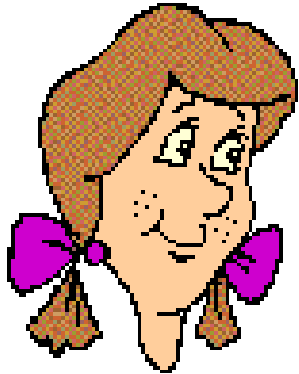
Don't use crypto algorithms for a different scope (even if a slightly different one!), or with different assumptions

CHAP



User Database	
...	...
flavia	mypass
...	...

CHAP and mutual authentication /1



CHALLENGE = 135623

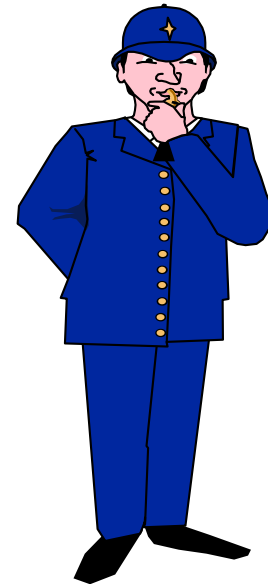
RESP: Flavia, H(sharedsecret, 135623)

ACK

CHALLENGE = 324567

RESP: servername, H(sharedsecret, 324567)

ACK



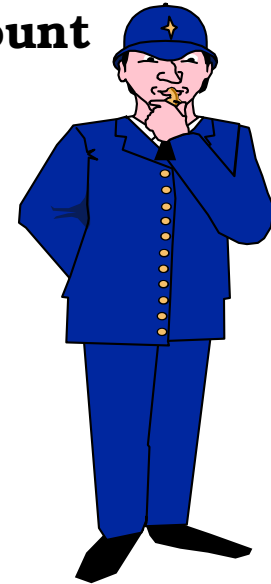
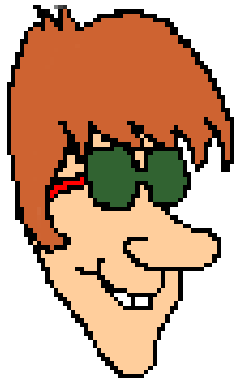
Usage of a shared
Secret... good idea,
Easy to manage!

Good idea??

CHAP and mutual authentication /2

Reflection attack

in PPP order of messages does not count



CHALLENGE = 135623

CHALLENGE = 135623

RESP: servername, H(sharedsecret, 135623)

ACK

RESP: client, H(sharedsecret, 135623)

ACK

VERY risky!!

Attacker:

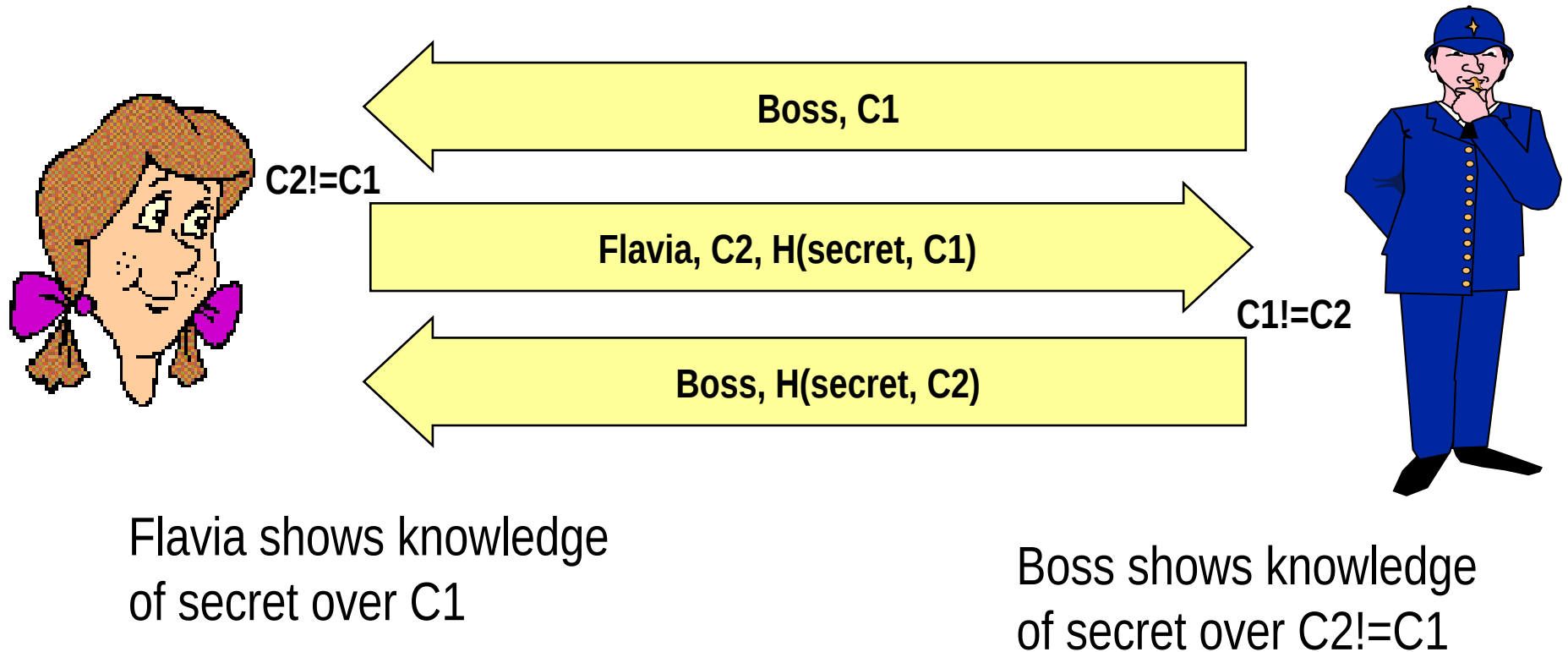
- replays server challenge
- accept computed resp
- uses resp to authenticate!!

Without any info on
real client !!

**Yes but this happened because you
used two unilateral authentication
protocols!**

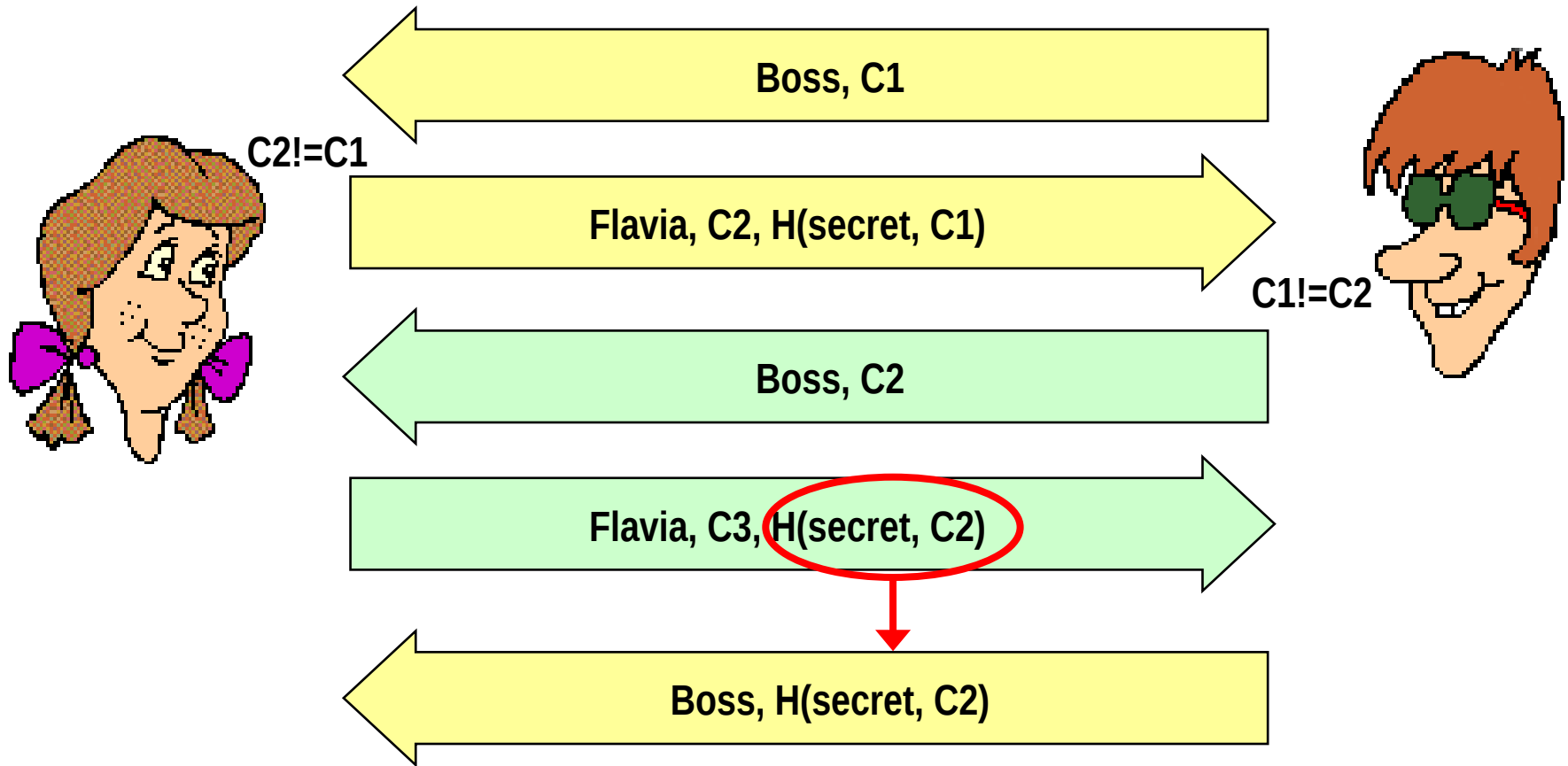
**Let's combine them into a single
Mutual authentication with
Challenge-Response**

Pippo's mutual authentication protocol (seriously, don't do this!)



Where is the flaw?

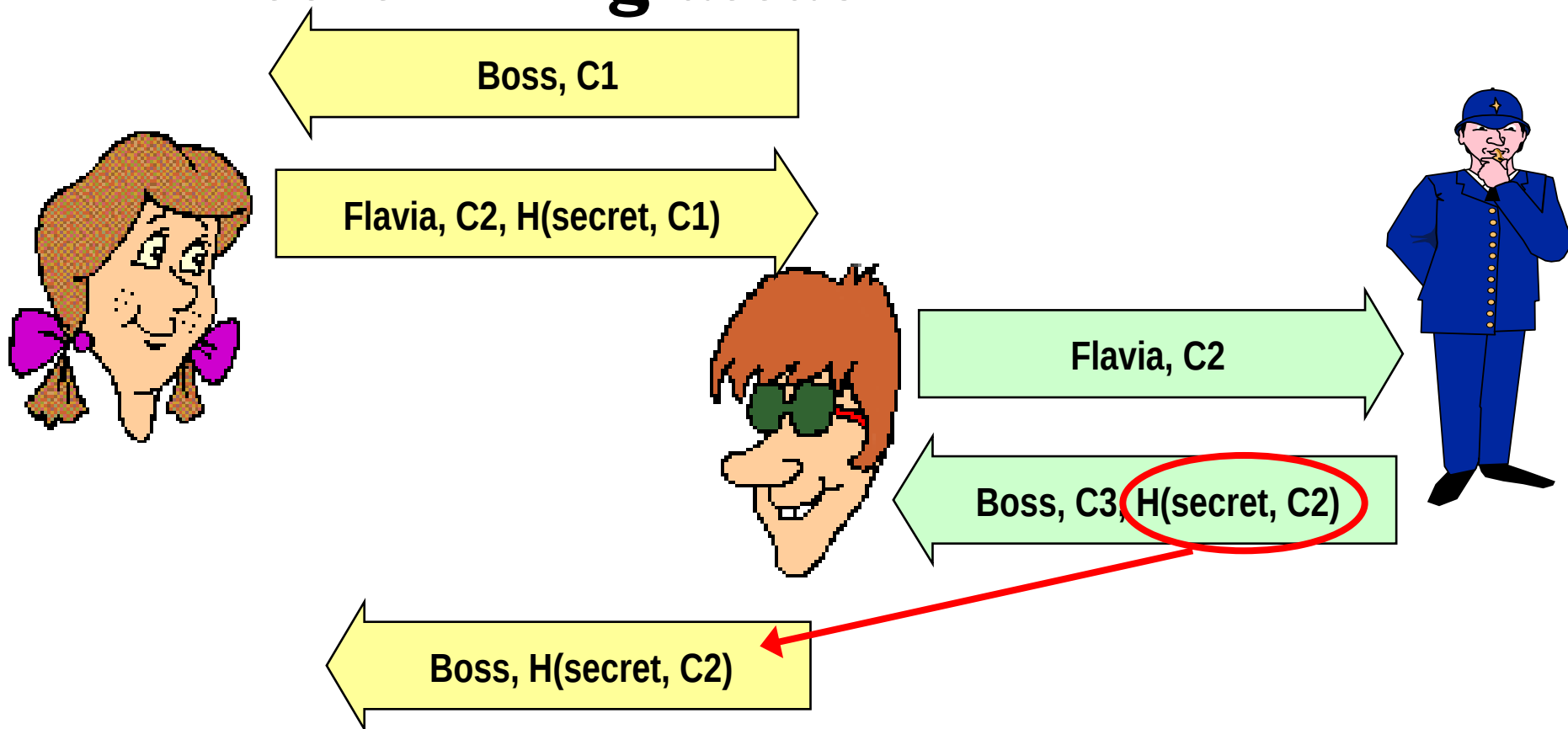
Reflection! Again!



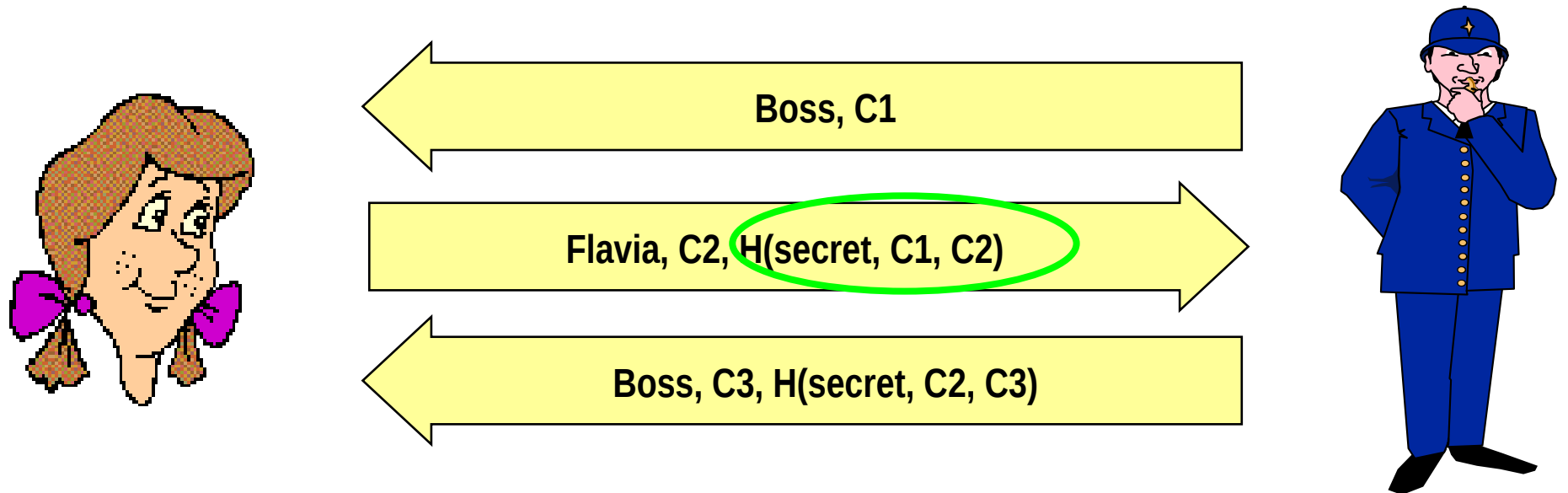
What if reflection is prevented by protocol status?

→ Attacker may use “other” party!

→ Intertwining attack

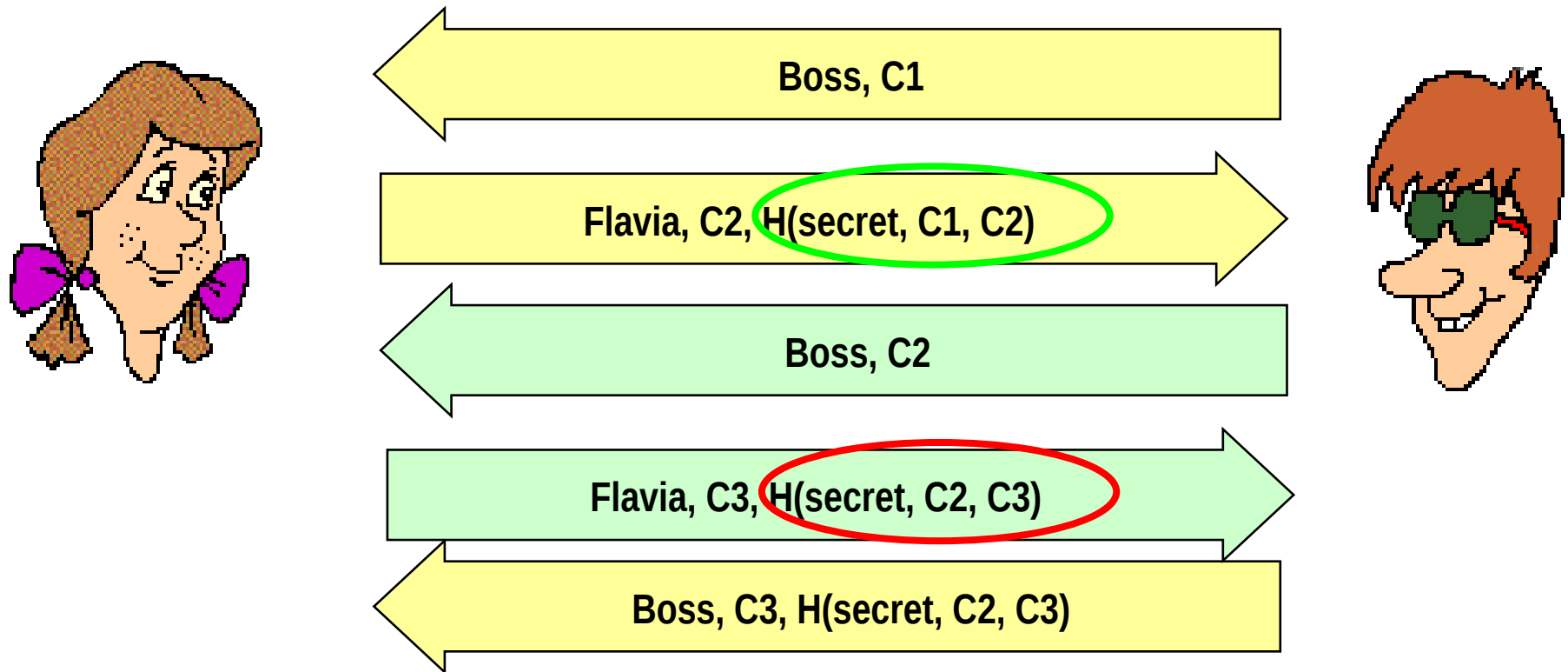


Let's try to fix this



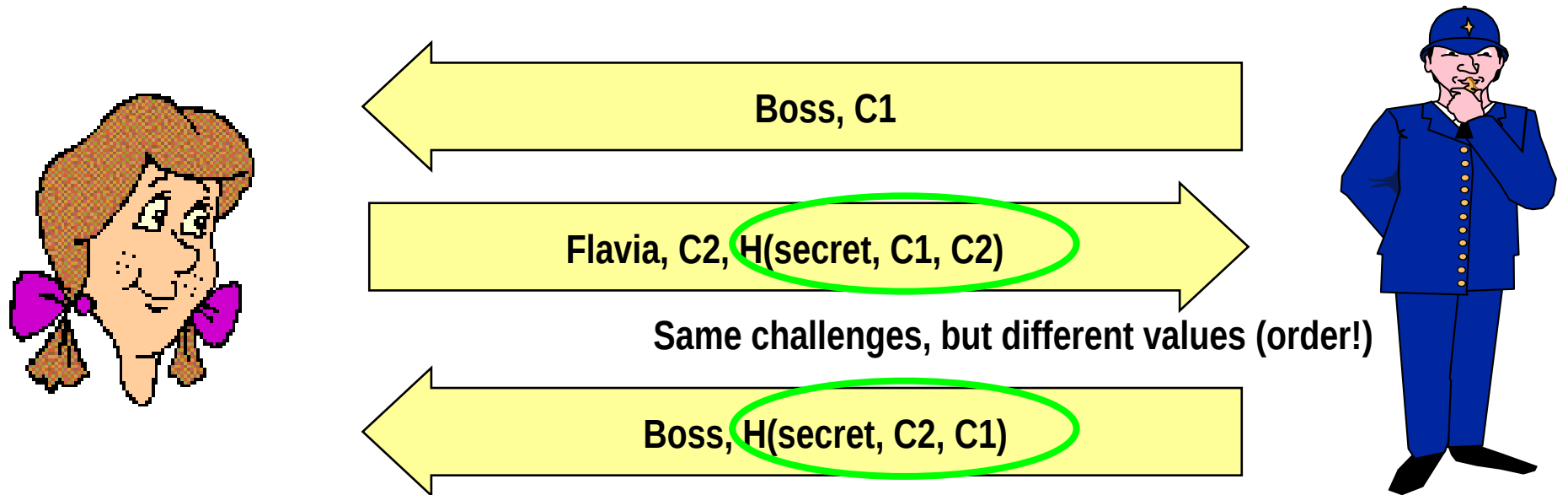
Chaining challenges! Add dependency
between challenges in same handshake

Does not work



Too many nonces!!

But if we minimize nonces...



No reflection possible anymore